

ESGC VERECHAGUINE A.K.

L2 Génie Informatique - Semestre 3

Théorie des Graphes - Séance 5

TD5 - ARBRE COUVRANT MINIMAL
Algorithme de Kruskal & Structure Union-Find

Durée : 45 minutes

Date : _____

Nom : _____

Prénom : _____

CONSIGNES : Travail individuel • Documents de cours autorisés • Calculatrice non nécessaire**EXERCICE 1 : DÉROULER L'ALGORITHME DE KRUSKAL**

■ 20 minutes • 10 points

OBJECTIF : Tracer l'algorithme de Kruskal étape par étape.

Appliquez l'algorithme de Kruskal sur le graphe G suivant.

GRAPHE G : Graphe non orienté pondéré avec 7 sommets (A, B, C, D, E, F, G) et 11 arêtes

Arête	Arête	Arête	Arête
A — B (poids 7)	B — C (poids 8)	C — E (poids 5)	E — G (poids 9)
A — D (poids 5)	B — D (poids 9)	D — E (poids 15)	F — G (poids 11)
A — C (poids 3)	B — E (poids 7)	D — F (poids 6)	

— **TRAVAIL PRÉLIMINAIRE** : Dessinez ce graphe sur votre feuille avant de commencer.**Question 1.1 (2 points) : Tri des arêtes**Listez toutes les arêtes triées par poids **croissant**.

Question 1.2 (6 points) : Exécution complète

Complétez le tableau en indiquant pour chaque arête si elle est acceptée ou rejetée.

#	Arête	Poids	Composantes de u et v	Action (Acceptée/Rejetée)	Pourquoi ?
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					

Légende : Acceptée = relie deux composantes différentes • Rejetée = créerait un cycle

Question 1.3 (2 points) : Résultat final

Arêtes du MST : { _____ , _____ , _____ , _____ , _____ , _____ }

Poids total du MST : _____

Nombre d'arêtes du MST : _____ (vérification : doit être $n-1 =$ _____)

EXERCICE 2 : STRUCTURE UNION-FIND

■ 15 minutes • 6 points

OBJECTIF : Comprendre et manipuler la structure Union-Find (DSU).

On considère **6 éléments** : {0, 1, 2, 3, 4, 5}. Initialement, chaque élément est son propre ensemble.

Question 2.1 (2 points) : Initialisation

Remplissez les tableaux **parent[]** et **rank[]** après l'initialisation (MAKE_SET pour chaque élément) :

i	0	1	2	3	4	5
parent[i]						
rank[i]						

Question 2.2 (3 points) : Suite d'opérations

Exécutez les opérations suivantes **avec union par rang** (le plus petit sous le plus grand). Après chaque UNION, dessinez l'état des arbres ou remplissez parent[].

Opération 1 : UNION(0, 1)

Opération 2 : UNION(2, 3)

Opération 3 : UNION(0, 2) – Attention au rang !

Opération 4 : UNION(4, 5)

Opération 5 : UNION(3, 5) – Quelles sont les racines ?

État final de parent[] après les 5 opérations :

i	0	1	2	3	4	5
parent[i]						
rank[i]						

Question 2.3 (1 point) : Opérations FIND

En utilisant l'état final (avec **compression de chemin**), donnez le résultat de :

FIND(1) = _____

FIND(5) = _____

CONNECTED(1, 5) = _____ (Vrai / Faux)

EXERCICE 3 : MODÉLISATION — RÉSEAU ROUTIER

■ 10 minutes • 4 points (+1 bonus)

OBJECTIF : Modéliser un problème réel et le résoudre avec Kruskal.

La commune d'Abomey-Calavi veut relier **6 villages** par des routes goudronnées. Le coût de construction entre chaque paire de villages (en millions FCFA) est donné ci-dessous. La commune veut **minimiser le coût total** tout en garantissant que tous les villages soient accessibles.

	Togba	Zoca	Godomey	Calavi	Akassato	Zinvié
Togba	—	4	8	10	7	14
Zoca	4	—	3	9	6	12

Godomey	8	3	—	5	11	13
Calavi	10	9	5	—	2	8
Akassato	7	6	11	2	—	6
Zinvié	14	12	13	8	6	—

Question 3.1 (2 points) : Modélisation

Combien de sommets a ce graphe ? _____

Combien d'arêtes a ce graphe ? _____ (indice : $n(n-1)/2$)

Pourquoi ce problème correspond-il exactement à un MST ?

Question 3.2 (2 points) : Résolution

Appliquez Kruskal et donnez les arêtes du MST :

Coût total minimal : _____ millions FCFA

Question 3.3 BONUS (1 point)

Si la route Calavi — Akassato (coût 2) est déjà construite, comment modifier votre approche ? Quel serait le nouveau coût total ?

BARÈME : 20 POINTS (+1 BONUS)

Exercice	Thème	Points
Exercice 1	Dérouler Kruskal	10 pts
Exercice 2	Union-Find (DSU)	6 pts
Exercice 3	Modélisation (routes)	4 pts (+1)

RAPPELS :

- **Arbre** = graphe connexe sans cycle. Un arbre à n sommets a exactement **$n-1$ arêtes**.
- **MST (Arbre Couvrant Minimal)** = arbre couvrant dont la somme des poids est MINIMALE.
- **Kruskal** = algorithme GLOUTON : trier les arêtes par poids croissant, ajouter si pas de cycle.
- **Union-Find** : FIND(x) = trouver le représentant ; UNION(x,y) = fusionner deux ensembles.
- **Compression de chemin** : lors de FIND, rattacher chaque noeud directement à la racine.
- **Union par rang** : rattacher l'arbre le plus petit sous le plus grand (rank = borne hauteur).
- **Complexité Kruskal** : $O(E \log E)$ pour le tri + $O(E \times \alpha(V))$ pour Union-Find = **$O(E \log E)$** .
- **Propriété de coupe** : l'arête de poids min traversant une coupe appartient au MST.
- **Propriété de cycle** : l'arête de poids max dans un cycle n'appartient pas au MST.